

The Starry Lake Platform

Ian Moffett

Lead Systems Architect
Apollo Telephone Laboratories

Sera5im

Systems Architect
Apollo Telephone Laboratories

1. Starry Lake (AT-0000MC02)

Starry Lake is a 64-bit RISC-like architecture that includes support for non-intrusive platform management technology.

2. General Architecture

This section describes the general architecture of Starry Lake chips.

2.1. Operating Modes

The Starry Lake platform has four operating modes:

1. Machine mode : Firmware-level execution mode
2. Supervisor mode : Operating system kernel mode
3. Management mode : Kernel adjacent, non destructive, readonly
4. User mode : User-level software

2.1.1. Machine Mode

Machine mode is the highest privilege level that system software may operate in, all processor cores are to start in machine mode upon reset. Machine mode is capable of writing, executing all of memory, control registers and is thus reserved for platform firmware. System software running in machine mode is responsible for hardware initialization and governs the environment that supervisor-level system software will operate in.

2.1.2. Supervisor Mode

Supervisor mode is the second highest privilege level that system software (typically an operating system kernel) may operate in.

System-software within this mode is responsible for governing the environment of user-mode applications.

2.1.3. User Mode

User mode is the least privileged level that system software may operate in, it is meant for user-level application software that cannot be trusted, software running in this mode does not have access to any control registers or certain instructions and must request services from the supervisor through a system-call mechanism.

2.1.4. Management Mode

Management mode serves to allow system management software to perform non-destructive semi-privileged instructions and to access memory specified by supervisor via per-process page tables. Software operating in this mode is to signal hints/events to the supervisor in an asynchronous manner through “Management Doorbells”. Software operating in management mode may access all model specific registers permitted by the supervisor (refer to Section 2.3).

2.2. Management Doorbells

Management Doorbells are a mechanism in which system software running in management mode is capable of sending asynchronous events to the supervisor. These events may be triggered by attempting to write model specific or control registers. Depending on the processor topology (whether single-cored or multi-cored), these events may be handled by the architecture in different ways. For single-cored systems, these events are equivalent to synchronous traps as the event loops back around to the same core.

2.3. Model Specific Registers

Model specific registers are specialized generation-specific registers used for configuring hardware features, modes and reading

performance counters, and platform state. They are to be accessed the specialized instructions: RDMSR and WRMSR.

2.4. Platform Mainbus

The Starry Lake platform utilizes a 300 MHz APTEL interconnect known as ARI. The mainbus utilizes a segmented ring topology. The ring is broken up into on-chip devices referred to as “segments”, the role of segments is to route bus traffic to adjacent segments.

2.4.1. Ring Segments

While an important role of ring segments is to route traffic between other segments, they also double as device endpoints. In other words, there is to be one bus slave connected per segment.

2.4.1.1. Ingress/Egress Channels and Tap Ports

Each segment is to have an ingress channel and an egress channel. The ingress channel routes traffic from the previous segment while the egress channel is to pass the ingress channel outwards to the next segment. The last segment in the ring is to route the egress channel to the ingress of the first segment, completing the ring.

Along with the ingress/egress channels of each segment exists a tap port. The tap port allows data to be injected onto the bus from a specific segment. Each segment is to utilize Time Division Multiplexing with a minimum slice period of two bus cycles to multiplex the ingress channel and the tap port onto the egress channel.

2.4.2. Framing

All data that is to be transmitted on the bus must be encapsulated within a “frame” which contains metadata about the payload. The frame is to contain an ARI tag which identifies the payload type (e.g., IRQs, memory requests, et cetera), a tag value of zero is reserved and must be treated as a no-operation by slave devices. Furthermore, when the tag is zero, all frame fields including the payload are to be zeroed.

Along with the tag exists a 4 bit sequence number which is to be incremented by the master per every transaction made. The payload within each frame is to be a minimum size of 128 bits.

2.4.3. Further Timing Requirements

When a master (such as an Integrated Memory Controller) wishes to drive the bus through its respective segment's tap port, it is to drive the bus for a minimum of 4 bus cycles before de-asserting the tap port to a low state (0).

2.5. Integrated Memory Controller

The application domain containing the processors is to implement an integrated memory controller responsible for generating bus transactions and deciding whether they should end up on the sideband bus (refer to Section 2.6) or the mainbus.

2.6. Sideband bus

Starry Lake implements a dedicated sideband bus in which specific linear address ranges corresponding to high-performance endpoints are to be accessed through. This reduces contention on the mainbus and allows for a dedicated fast-path interconnect.

2.7. Paging and Protection

This section describes the memory protection architecture of Starry Lake implementations.

2.7.1. Pagelets

Starry Lake supports specialized 8-byte pagelets, its usage is out of scope of this document and is up to the implementation of system software. Pagelets introduce an L bit into the last-level page table for marking 8-byte regions on an 8-byte boundary as pagelets.

2.7.2. Standard Pages

Starry Lake has a standard page size of 4K to be mapped upon equally sized physical memory frames.

2.8. Platform Memory Map

This section describes the platform memory map:

```
0x00000000000000 - 0x000000002000 : Boot Rom
0x000000002000 - 0x000000005000 : Reserved
0x000000005000 - 0x000000105000 : System-level-cache
```

2.9. Processor Reset State

Upon reset, the processor is to initialize G0-G6, SP, FP, RA, A0-A7, LST, TLS and PC to zero. The program counter (PC) starting at zero begins execution in the platform boot ROM in which platform firmware takes control in machine mode.

3. Instruction-set Architecture

This guide describes the AT Instruction-Set-Architecture for APTEL silicon, developers of system software are encouraged to review this section and grasp a firm understanding.

3.1. Terminology

- RO : Readonly
- RW : Read/write
- WARL : Write any, read legal
- WPRI : Reserved writes, preserve values, reads ignored values

3.2. Processor registers

ATISA compliant silicon is to implement the registers that are listed below:

NAME	PURPOSE	ID

G0	: General purpose register 0	: 0x00
G1	: General purpose register 1	: 0x01
G2	: General purpose register 2	: 0x02
G3	: General purpose register 3	: 0x03
G4	: General purpose register 4	: 0x04
G5	: General purpose register 5	: 0x05
G6	: General purpose register 6	: 0x06
SP	: Stack pointer register	: 0x07
FP	: Frame pointer register	: 0x08
RA	: Return address register	: 0x09
A0	: Argument register 0	: 0x0A
A1	: Argument register 1	: 0x0B
A2	: Argument register 2	: 0x0C
A3	: Argument register 3	: 0x0D
A4	: Argument register 4	: 0x0E
A5	: Argument register 5	: 0x0F
A6	: Argument register 6	: 0x10
A7	: Argument register 7	: 0x11
LST	: Local state pointer	: 0x12
TLS	: Thread local storage	: 0x13

- Registers G0 - G6 are general purpose and can be used in however way.
- Register SP is used for pointing to the stack top.
- Register FP is used for call-stack linking and local variable addressing.
- Register RA stores the return address, ret is an indirect branch to RA
- Registers A0 - A7 stores arguments in ascending order.
- Register LST stores kernel specific state per-core.
- Register TLS stores per-core thread local storage.



3.2.1. Control registers

Starry Lake implementations must provide a set of always-supported control registers. Unlike Model Specific Registers (refer to Section 2.3) which provide chip specific knobs, every control register fields are to supported by all Starry Lake implementations.

Below is a table describing a list of control registers acessible by system software:

NAME	PURPOSE	ACCESS	ID
MREV	Chip revision	R0	0x00

Control registers prefixed with ‘M’ are accessible in machine mode only, attempting to access machine-mode registers in a lower privilege space will result in a processor exception. Similar behavior occurs with S-mode (‘S’ prefixed) registers.

3.3. Instruction listing

ATISA mandates 32-bit fixed-width instructions of varying types, below is a listing of instructions, opcodes and their respective types:

MNEMONIC	BRIEF	OPCODE	TYPE
RSVD	Reserved	0x00	[R]
NOP	No-operation; waste cycle	0x01	[A]
WFI	Wait-for-interrupt; halt	0x02	[A]
SPW	Spin-wait hint	0x03	[A]
RDMSR	Read MSR w/ ID in G0 to G1	0x04	[A]
WRMSR	Write MSR w/ ID in G0 from G1	0x05	[A]



3.3.1. Instruction types

A-type instructions:

